

Problem Set 1

Problem 1.1

a

- The curly braces denote a region of code in the program, specifically within a function or conditional. It makes sense to surround the body of a function with curly braces because it restricts the scope of the block of code within the program, along with making the program modular and readable.

b

- `7` is an integer literal, `"7"` is a string value due to the double quotations, and `'7'` is a character due to the use of apostrophe marks.

c

- The statement can be rewritten as `10.0+(2.0/((3.0-2.0)*2.0))` to ensure that the answer will be `11.0`.

Problem 1.2

- The answer would be 18.0.
- The answer would be 2.0.
- The answer would be 18.0.
- The answer would be 2.0.

Problem 1.3

- The command used to compile my program is `gcc -c -Wall -O0 assignment.c -o assignment.o`. The command used to execute my program with `gdb` was `gdb assignment.o`. The following is the output of my program:

```
└─$ █ █ █ /mnt/c/U/s/Doc/LearningC/Week_001/Assignment █ █ ✓ █ 37s █ █ 20:41:51 █ █ █
└─$ gcc assignment.o -o assignment

└─$ █ █ █ /mnt/c/U/s/Documents/LearningC/Week_001/Assignment █ █ ✓ █ 20:42:19 █ █ █
└─$ ./assignment
Hello, 6.087 students

└─$ █ █ █ /mnt/c/U/s/Documents/LearningC/Week_001/Assignment █ █ ✓ █ 20:42:22 █ █ █
└─$
```

Problem 1.4

```
#include <stdio.h>
#define MSG1 "All your base are belong to us!"
int main(void) {
    const char msg[] = MSG1;
    puts(msg);
    return 0;
}
```

Problem 1.5

a

- Preprocessor directives and statements should NOT have a semicolon at the end of them.

```
#include <stdio.h>
```

b

- The proper type was not used for the argument which would result in a difference of types inputted.

```
int function(int arg1) {  
    return arg1 - 1;  
}
```

C

- The issue with this is that macros are often used globally in each C program along with macros not needing an `=`.

```
#define MESSAGE "Happy new year!"  
puts(MESSAGE);
```